

ROS2 Day3 AI 사용 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-11

목차

1. AI 사용 목적
2. Task01에서의 AI 활용
3. Task02에서의 AI 활용
4. AI 사용 과정에서 느낀 점

1. AI 사용 목적

이번 ROS2 Day3 과제를 진행하면서 AI는 Launch와 YAML을 이용한 Parameter 관리 방법과 ROS2 노드와 Qt 프로그램을 연결하는 과정을 이해하기 위해 사용하였다.

수업 자료를 먼저 확인한 뒤 YAML 파일의 값을 C++ 코드에서 어떻게 가져오는지, 여러 노드를 하나의 Launch 파일에서 어떻게 실행하는지와 같이 이해되지 않는 부분을 AI에게 질문하였다.

또한 Task02에서는 Traffic Light Node와 Vehicle Node를 Topic으로 연결하고, Qt에서는 두 노드에서 전달받은 값을 시각적으로 표시하는 구조를 구현하는 과정에서 AI를 활용하였다. Qt Designer에서 작성한 UI를 C++ 코드와 연결하거나 빌드 과정에서 오류가 발생했을 때에도 오류의 원인을 확인하기 위해 질문하였다.

AI의 답변을 그대로 사용하는 것보다 설명을 확인한 뒤 실제로 코드를 빌드하고 실행하여 결과가 정상적으로 나오는지 확인하였다. 예상한 결과와 다르게 동작하는 경우에는 실행 결과와 오류 메시지를 다시 확인하면서 필요한 부분을 수정하였다.

2. Task01에서의 AI 활용

Task01을 진행하면서 기존 Day1 Task03 프로그램에 Parameter, YAML, Launch 기능을 추가하였다. 처음에는 YAML 파일에 값을 작성한 뒤 해당 값을 C++ 코드에서 어떤 과정으로 불러오는지 정확히 이해되지 않아 AI에게 질문하였다.

AI의 설명을 통해 C++ 코드에서는 먼저 `declare_parameter()`를 이용하여 Parameter를 선언하고, `get_parameter()`를 이용하여 실제 값을 가져와야 한다는 것을 이해하였다. 또한 YAML 파일에 작성한 값은 Launch 파일에서 해당 노드의 Parameter로 전달하도록 구성해야 한다는 것도 확인하였다.

이를 바탕으로 `pen_r`, `pen_g`, `pen_b`, `pen_width`를 Parameter로 선언하고, `setPen()` 함수에서 값을 가져와 turtlesim의 `set_pen` 서비스에 전달하도록 구성하였다.

또한 Launch 파일을 이용하여 turtlesim과 Task01 노드를 동시에 실행하는 과정에서 키보드 입력을 받는 Task01 노드가 정상적으로 입력을 받지 못하는 문제가 있었다. 이 문제에 대해 AI에게 질문하면서 키보드 입력을 사용하는 노드를 별도의 터미널에서 실행하도록 구성하는 방법을 확인하였다.

이후 Launch 파일을 실행하여 turtlesim과 Task01 노드가 함께 실행되는 것을 확인하였으며, YAML 파일에서 RGB 값과 선의 굵기를 변경한 뒤 다시 실행하여 변경된 Parameter가 정상적으로 적용되는 것을 확인하였다.

YAML에 적은 값을 C++에서 어떻게 불러와? declare_parameter랑 get_parameter 차이가 잘 이해가 안 돼

`declare_parameter()` 는 “이 노드에서 이런 이름의 파라미터를 사용할 거야”라고 등록하는 역할이고, `get_parameter()` 는 현재 설정된 값을 실제로 가져오는 역할이야.

예를 들어 C++에서 이렇게 선언하면:

```
</> C++  
  
declare_parameter("pen_r", 255);  
declare_parameter("pen_g", 0);  
declare_parameter("pen_b", 0);  
declare_parameter("pen_width", 3);
```

`pen_r`, `pen_g`, `pen_b`, `pen_width` 라는 파라미터를 사용할 수 있게 돼. 뒤의 숫자는 YAML에서 값이 따로 안 들어왔을 때 사용할 기본값이야.

그다음 실제 값을 쓸 때는:

```
</> C++  
  
int r = get_parameter("pen_r").as_int();  
int g = get_parameter("pen_g").as_int();  
int b = get_parameter("pen_b").as_int();  
int width = get_parameter("pen_width").as_int();
```

이렇게 가져와.

YAML에 예를 들어:

```
</> YAML
```

3. Task02에서의 AI 활용

Task02에서는 Traffic Light Node, Vehicle Node, Qt GUI를 각각 구성하고 Topic을 이용하여 서로 연결하였다.

처음에는 신호등의 상태 변경과 차량의 이동, Qt 화면 출력을 하나의 프로그램에서 모두 처리해야 한다고 생각하였다. 하지만 AI에게 구조를 질문하면서 신호등 상태는 `traffic_light_node`, 차량의 움직임은 `vehicle_node`, 화면 표시는 Qt 프로그램으로 역할을 나누는 방법을 이해하였다.

`traffic_light_node`에서는 RED, GREEN, YELLOW 상태를 변경하면서 `/traffic_light` Topic을 발행하고, `vehicle_node`에서는 해당 Topic을 구독하여 GREEN 상태에서는 이동하고 YELLOW 상태에서는 속도를 줄이며 RED 상태에서는 정지하도록 구성하였다.

Qt 프로그램에서는 `/traffic_light`과 `/vehicle_position` Topic을 구독하여 현재 신호 상태와 차량 위치를 화면에 표시하도록 하였다. 이를 통해 Qt 자체에서 차량의 이동 여부를 판단하지 않고 ROS2 노드에서 계산된 값을 시각적으로 표현하는 구조를 이해할 수 있었다.

또한 Qt Designer에서 작성한 mainwindow.ui 파일을 C++ 코드와 연결하고 빌드하는 과정에서 UI 파일을 찾지 못하는 오류가 발생하였다. 오류 내용을 AI에게 질문하여 CMake에서 Qt의 AUTOUIC 설정과 UI 파일 경로를 확인하였고, 수정 후 다시 빌드하여 Qt 프로그램이 정상적으로 실행되는 것을 확인하였다.

마지막으로 YAML 파일에서 신호등의 점등 시간과 vehicle_speed 값을 변경한 뒤 프로그램을 다시 실행하여 신호 유지 시간과 차량 속도에 변경된 값이 적용되는 것을 확인하였다.

traffic_light_node에서 신호를 보내고 vehicle_node가 그걸 받아서 움직이게 하려면 토픽을 어떻게 연결해야 해? Qt에서는 그냥 화면만 보여주면 되는 거야?

traffic_light_node에서 신호 상태를 **Publisher**로 발행하고, vehicle_node에서 같은 토픽을 **Subscriber**로 구독하면 돼.

예를 들어 traffic_light_node에서 /traffic_light 토픽으로 현재 상태를 보내는 식이야.

```
</> C++  
pub_ = create_publisher<std_msgs::msg::String>("/traffic_light", 10);
```

그리고 현재 신호를:

```
</> C++  
std_msgs::msg::String msg;  
msg.data = state_;  
pub_->publish(msg);
```

이렇게 발행해.

vehicle_node에서는 같은 /traffic_light 토픽을 구독하면 돼.

```
</> C++  
sub_ = create_subscription<std_msgs::msg::String>(  
    "/traffic_light",  
    10,  
    std::bind(  
        &Vehicle::lightCallback,  
        this,  
        std::placeholders::_1
```

4. AI 사용 과정에서 느낀 점

이번 ROS2 Day3 과제를 진행하면서 AI는 코드를 바로 작성하기 위한 용도보다는 새로운 기능의 구조를 이해하고, 구현 과정에서 발생한 문제의 원인을 확인하는 데 도움이 되었다.

특히 YAML 파일에 작성한 Parameter가 Launch 파일을 거쳐 C++ 노드에 전달되는 과정과 여러 ROS2 노드를 Topic으로 연결한 뒤 Qt에서 결과를 시각화하는 과정은 처음에는 구조가 복잡하게 느껴졌다. 각각의 기능을 나누어 질문하면서 어떤 파일과 노드가 어떤 역할을 하는지 이해할 수 있었다.

또한 AI의 설명만 보고 정상적으로 구현되었다고 판단하지 않고 직접 프로그램을 빌드하고 실행하면서 결과를 확인하는 과정이 중요하다는 것을 다시 느꼈다. Task01에서는 YAML 값을 변경하여 선의 색상과 굵기가 실제로 변경되는지 확인하였고, Task02에서는 신호등 상태에 따라 차량이 이동하거나 정지하는지와 Qt 화면이 Topic 값을 정상적으로 표시하는지 직접 확인하였다.

빌드 오류가 발생했을 때에도 오류 메시지를 확인하고 AI의 설명과 현재 프로젝트의 파일 구조를 비교하면서 수정하였다. 앞으로도 AI를 사용할 때에는 먼저 수업 자료와 현재 코드를 확인하고, 이해되지 않는 개념이나 오류를 질문한 뒤 실제 실행 결과를 통해 다시 확인하는 방식으로 활용할 생각이다.